MaintainIQ: An End-to-End Machine Learning System for Industrial Predictive Maintenance and Explainable Failure Risk Assessment

Academic Capstone Project Final Report
Course: Applied Machine Learning & Predictive Analytics
Academic Term: 2026
Repository & Implementation: MaintainIQ Project

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Literature Review & Theoretical Framework
4. Dataset Description and Synthetic-Data Limitations
5. Data Cleaning, Feature Engineering, and Leakage-Safe Preprocessing
6. Methodology and Experimental Setup
7. Model Comparison, Performance Benchmarking, and Results
8. Model Explainability and Feature Attribution Analysis
9. Streamlit Dashboard Architecture, Deployment, and User Workflow
10. Discussion, Limitations, and Future Directions
11. Conclusion
12. References
13. Appendix: Reproducibility and System Commands

1. Abstract

Unplanned machinery downtime represents one of the largest drivers of unrecoverable operational expenditures and safety hazards across modern manufacturing facilities. Traditional maintenance paradigms—reactive (run-to-failure) and preventive (calendar-based)—consistently fail to balance maintenance overhead against catastrophic equipment destruction. This project presents **MaintainIQ**, a modular, leakage-safe machine learning system designed to monitor continuous sensor telemetry from industrial milling operations and assess equipment failure risk prior to catastrophic failure.

Using the **UCI AI4I 2020 Predictive Maintenance Dataset** (10,000 records, 14 attributes), MaintainIQ benchmarks four distinct supervised classification algorithms: **Logistic Regression** (linear baseline), **Random Forest** (bagging ensemble), **XGBoost** (extreme gradient boosting), and **CatBoost** (categorical gradient boosting). The dataset exhibits severe class imbalance, with positive failure events occurring in only **3.39%** of observations. Addressing this challenge, all data preprocessing and categorical encoding transformations were embedded in a strict scikit-learn pipeline fit exclusively on training data following a reproducible 80/20 stratified train/test split (`random_state=42`), eliminating all forms of data leakage.

Given the acute positive-class rarity, models were evaluated and selected using **Precision-Recall Area Under the Curve (PR-AUC / Average Precision)** alongside Recall, Precision, and F1-score, rather than deceptive scalar accuracy or unweighted ROC-AUC. Empirical evaluation on the held-out test split (2,000 samples, 68 failures) demonstrated that **XGBoost** configured with training-derived scale-positive weighting achieved superior operational performance: **PR-AUC of 0.8283**, **F1-score of 0.7376**, **Recall of 76.47%** (catching 52 of 68 test failures), **Precision of 71.23%**, and an inference latency of **1.38 µs per sample**. Random Forest achieved the highest precision (75.41%, 15 false alarms), while CatBoost attained the highest raw recall (85.29%, 58 of 68 failures detected).

To resolve the industrial black-box barrier, MaintainIQ integrates dual-level Explainable AI (XAI): global permutation feature importance (measuring true degradation in PR-AUC across 10 repeats) and local instance feature attributions via TreeSHAP (`TreeExplainer`). Global evaluation established that cutting torque (mean PR-AUC drop: 0.5985), ambient air temperature (drop: 0.5062), and spindle rotational speed (drop: 0.4183) govern model predictions, aligning with the physical laws of power overload, heat dissipation breakdown, and mechanical overstrain. The complete pipeline is operationalized through an interactive, multi-tab Streamlit dashboard supporting single-machine telemetry risk scoring, batch CSV evaluation with automatic leakage exclusion, benchmark visualization, and explainability inspection. Finally, we explicitly emphasize that the AI4I 2020 dataset is derived from a phenomenological physical simulator; consequently, while our results validate pipeline safety and algorithmic capability, they do not establish uncalibrated real-world industrial reliability guarantees.

---

2. Introduction and Problem Statement

2.1 The Industrial Maintenance Paradigm

In competitive industrial manufacturing, the operational integrity of rotating machinery and machine tools (such as CNC milling machines, lathes, and high-speed spindles) directly dictates factory

throughput, product quality, and workplace safety. Historically, plant operations have relied upon two maintenance paradigms:

1. **Reactive Maintenance (Run-to-Failure)**: Machinery is operated without intervention until catastrophic breakdown occurs. While this strategy eliminates pre-failure servicing costs, it induces exorbitant direct costs: emergency technician call-outs, severe secondary mechanical damage (e.g., a snapped cutter damaging spindle bearings or workpiece stock), worker hazards, and cascading assembly line halts.
2. **Preventive Maintenance (Time- or Cycle-Based)**: Servicing, lubrication, and tool replacement are conducted according to conservative factory schedules or manufacturer-recommended cycle counts. While preventive maintenance reduces catastrophic failures, it is inherently inefficient: components with substantial remaining useful life (RUL) are prematurely discarded, and maintenance resources are expended on machines operating in peak condition. Furthermore, unpredicted infant-mortality component defects or atypical operational stresses can still cause catastrophic failures between scheduled maintenance windows.

Predictive Maintenance (Condition-Based) leverages real-time sensory telemetry streams (including thermal gradients, rotational speeds, load torques, and cumulative cutter contact time) paired with statistical machine learning algorithms [1], [2]. The predictive objective is to monitor continuous physical degradation signals and generate timely, probabilistic risk scores that allow maintenance engineers to intervene only when structural fatigue or impending failure is statistically probable.

2.2 Core Technical Challenges

Translating predictive maintenance from theoretical formulation into production engineering faces four foundational challenges:

- **Acute Class Imbalance**: In well-engineered industrial machinery, failure is an extreme outlier event, typically occurring in less than 5% of operational cycles. Standard classification algorithms optimizing for zero-one loss or accuracy succumb to majority-class bias, achieving deceptively high accuracy (>95%) while failing to predict a single critical failure [3], [6].
- **Data Leakage in Feature Pipelines**: Industrial telemetry preprocessing frequently suffers from methodological leakage: fitting scalers or imputers across entire datasets prior to splitting, or applying synthetic oversampling techniques (such as SMOTE) across evaluation sets. When evaluated under strict production conditions, these leaky pipelines collapse [4].
- **The "Black-Box" Usability Chasm**: Modern high-performing non-linear estimators (e.g., gradient boosted decision trees) produce complex decision surfaces. Shop-floor maintenance technicians and reliability engineers routinely reject automated alert systems when the algorithm cannot provide clear, physically verifiable reasoning for an alert [1], [4].
- **Real-Time Operational Latency**: Predictive systems deployed in industrial IoT (IIoT) edge environments or plant-wide supervisory control and data acquisition (SCADA) systems require microsecond-level inference latency to score telemetry batches continuously without creating processing bottlenecks.

2.3 Problem Statement and Project Scope

The objective of the **MaintainIQ** project is to design, implement, rigorously evaluate, and deploy an end-to-end, leakage-safe machine learning system that:

1. Ingests industrial milling machine telemetry from the UCI AI4I 2020 dataset and cleans operational records without data leakage.
2. Separates the six approved sensor feature variables from target labels and ground-truth failure mode breakdown indicators to prevent target leakage.
3. Benchmarks four diverse classification algorithms (Logistic Regression, Random Forest, XGBoost, CatBoost) under cost-sensitive class reweighting on an 80/20 stratified split (`random_state=42`).
4. Evaluates performance using threshold-independent **PR-AUC**, Precision, Recall, and F1-score rather than misleading overall accuracy.
5. Implements model-agnostic permutation feature importance and local game-theoretic TreeSHAP feature attributions.
6. Deploys the best-performing pipeline through an interactive Streamlit web dashboard supporting single-record inference, batch CSV upload scoring, model metric inspection, and explainability visualization.

---

3. Literature Review & Theoretical Framework

A survey of contemporary literature reveals significant advances and ongoing debates in applying machine learning to industrial predictive maintenance. MaintainIQ's system design is anchored in nine key peer-reviewed publications: five recent studies published between 2021 and 2026, supplemented by four foundational works.

```
+-------------------------------------------------------------------------------------------+
|                                MAINTAINIQ THEORETICAL FOUNDATION                          |
+-----------------------------------+-------------------------------------------------------+
| Foundational Paradigms            | Recent Advances (2021–2026)                           |
| - Matzka (2020) [1]: AI4I Dataset | - Alnahhal et al. (2026) [3]: AI4I Imbalance Handling |
| - Carvalho et al. (2019) [2]: PdM | - Theissler et al. (2021) [4]: PdM Challenges & XAI   |
| - Saito & Rehmsmeier (2015) [8]: PR| - Marmoucha et al. (2026) [5]: Multicriteria Selection |
| - Lundberg et al. (2020) [9]: SHAP | - Chatterjee & Byun (2023) [6]: Hybrid Tree Ensembles |
|                                   | - Abidi et al. (2022) [7]: Industry 4.0 PdM Architectures |
+-----------------------------------+-------------------------------------------------------+
```

3.1 Foundational Literature

- **Matzka (2020)** [1] introduced the **AI4I 2020 Predictive Maintenance Dataset** at the IEEE AI4I conference, generating 10,000 synthetic records simulating milling machine physics. Matzka demonstrated that while complex non-linear models achieve high predictive accuracy on tabular telemetry, shop-floor operators require transparent decision rules to trust automated maintenance interventions. The paper established cutting torque, rotational speed, and tool wear contact time as primary failure indicators.
- **Carvalho et al. (2019)** [2] conducted an extensive systematic review spanning over 200 predictive maintenance studies in *Computers & Industrial Engineering*. The authors established that tree-based ensemble algorithms (Random Forest and gradient boosting) consistently outperform linear models, support vector machines, and standalone neural networks on structured tabular sensor telemetry. Their superiority stems from natural resistance to sensor noise, invariance to feature scale disparities, and the capability to model complex physical interactions without requiring massive training datasets.
- **Saito & Rehmsmeier (2015)** [8] published a definitive mathematical proof in *PLOS ONE* demonstrating that the Receiver Operating Characteristic (ROC) plot presents an overly optimistic and deceptive picture of classifier performance on heavily imbalanced datasets. Because the

False Positive Rate ($FPR = FP/(FP + TN)$) has the massive negative class in its denominator, hundreds of false alarms generate only a minor increase in FPR. In contrast, Precision-Recall (PR) curves isolate the minority positive class, ensuring that model selection reflects true alarm reliability.

- **Lundberg et al. (2020)** [9] published **TreeSHAP** (`TreeExplainer`) in *Nature Machine Intelligence*, formulating a low-order polynomial-time algorithm ($O(TLD^2)$) to calculate exact game-theoretic Shapley additive feature attributions for tree ensembles. TreeSHAP guarantees local accuracy, missingness, and consistency, overcoming the heuristic approximations of legacy attribution methods and enabling verifiable feature attribution for individual predictions.

3.2 Recent Studies (2021–2026)

- **Alnahhal et al. (2026)** [3] evaluated 20 configurations across four imbalance-handling approaches on the UCI AI4I 2020 dataset in *Computation*. The study proved that gradient boosting algorithms exhibit inherent resilience on tabular industrial data, and that cost-sensitive algorithmic reweighting (penalizing minority misclassifications) preserves true feature distributions significantly better than aggressive synthetic oversampling (such as SMOTE), which introduced synthetic artifacts near operational decision boundaries.
- **Theissler et al. (2021)** [4] surveyed industrial predictive maintenance implementations in *Reliability Engineering & System Safety*. The authors identified two primary bottlenecks to industrial deployment: extreme class imbalance (< 5% failure incidence) and operator resistance to black-box systems. The paper emphasized that post-hoc explainability is essential for bridging the gap between algorithmic predictions and physical maintenance protocols.
- **Marmoucha et al. (2026)** [5] proposed an Entropy-weighted TOPSIS multicriteria decision-making framework in *Applied Sciences*, benchmarking Logistic Regression, Random Forest, XGBoost, and LightGBM across the UCI AI4I 2020 dataset and a real-world metro air compressor dataset (MetroPT). The authors proved that gradient boosted trees dominate linear models across both datasets, balancing high failure detection rates against low false positives while sustaining microsecond inference latencies.
- **Chatterjee & Byun (2023)** [6] investigated extreme class imbalance in industrial condition monitoring in *Engineering Applications of Artificial Intelligence*. The authors demonstrated that uncalibrated classifiers optimize raw accuracy by predicting healthy states, and confirmed that tree-based ensembles configured with minority-class weighting achieve high recall without causing prohibitive false-alarm rates.
- **Abidi et al. (2022)** [7] formulated an end-to-end predictive maintenance planning architecture in *Sustainability*, demonstrating that systematic preprocessing, feature normalization, and disciplined classification pipelines yield substantial reductions in unplanned plant downtime.

3.3 Identified Research Gaps and MaintainIQ Contributions

Contemporary literature frequently suffers from three recurring deficiencies:

1. *Pervasive Preprocessing Leakage*: Preprocessing transformations (scaling, encoding, imputing) are frequently fit on the full dataset before splitting.
2. *Deceptive Evaluation Reporting*: Studies routinely cite >98% accuracy on imbalanced data without reporting PR-AUC or complete confusion matrices.

3. *Lack of Usable Deployment*: Theoretical models are rarely packaged into interactive applications accessible to maintenance operators.

MaintainIQ addresses each gap by enforcing a strict, leak-free scikit-learn pipeline, benchmarking four diverse models using PR-AUC and cost-sensitive reweighting, providing dual-layer XAI, and deploying an interactive Streamlit application.

---

4. Dataset Description and Synthetic-Data Limitations

4.1 Dataset Provenance & Structure

MaintainIQ utilizes the **UCI AI4I 2020 Predictive Maintenance Dataset** (Dataset ID: 601), authored by Stephan Matzka [1] and hosted by the UCI Machine Learning Repository. The dataset contains **10,000 rows** (operational observations) and **14 columns**, reflecting continuous condition monitoring of an industrial CNC milling machine.

UCI AI4I 2020 DATASET ARCHITECTURE		
Attribute Category	**Variable Name**	**Operational Definition**
Row Identifiers (Excluded)	UDI	Unique row identifier (1 to 10,000)
	Product ID	Variant letter (L/M/H) + serial number
Approved Feature Inputs (6 Variables)	Type	Quality variant (L: 50%, M: 30%, H: 20%)
	Air temperature [K]	Ambient room temperature (mean ~300.0 K)
	Process temperature [K]	Cutting process temperature (~310.0 K)
	Rotational speed [rpm]	Spindle velocity (calculated from power)
	Torque [Nm]	Cutting resistance torque (mean ~39.98 Nm)
	Tool wear [min]	Cumulative cutter usage time (0–253 min)
Target Label	Machine failure	Binary ground truth (0 = Normal, 1 = Fail)
Failure Mode Breakdown Indicators (Excluded)	TWF (Tool Wear Failure)	Tool wear exceeds failure threshold
	HDF (Heat Dissipation Fail)	Thermal gradient ΔT < 8.6 K at low speed
	PWF (Power Failure)	Spindle power < 3500 W or > 9000 W
	OSF (Overstrain Failure)	Torque × Tool wear exceeds limit
	RNF (Random Failure)	Stochastic hardware failure (~0.1%)

4.2 Approved Features vs. Leakage Exclusion

To guarantee absolute methodological validity, MaintainIQ enforces strict column separation in `src/preprocessing.py`:

- **Approved Feature Inputs**: Only six operational variables (`Type`, `Air temperature [K]`, `Process temperature [K]`, `Rotational speed [rpm]`, `Torque [Nm]`, `Tool wear [min]`) are provided to the learning algorithms.
- **Identifier Exclusions**: `UDI` and `Product ID` are dropped because arbitrary sequential serial numbers contain no physical condition signals and would allow decision trees to memorize row locations.
- **Failure Mode Breakdown Exclusions**: The five columns `TWF`, `HDF`, `PWF`, `OSF`, and `RNF` are detailed diagnostic breakdown indicators. Because these columns are subsets of `Machine failure` and are only known *after* a failure occurs, including them as model inputs would represent severe **target leakage**, rendering evaluation invalid.

4.3 Class Imbalance Profile

Across the 10,000 records:

- **Normal Operations (Class 0)**: 9,661 records (**96.61%**)
- **Machine Failures (Class 1)**: 339 records (**3.39%**)

This establishes an imbalance ratio of approximately **28.5:1**. A trivial majority-class classifier predicting zero failures achieves 96.61% accuracy while remaining 100% blind to machine breakdowns.

4.4 Synthetic-Data Nature and Industrial Limitations

A critical contribution of this academic report is an explicit disclosure of dataset origin and limitations:

1. **Simulator Origin**: The AI4I 2020 dataset was generated via a phenomenological physical simulator that calculates mathematical curves for temperature transfer, rotational resistance, and cutter wear [1].
2. **Absence of Real-World Stochasticity**: Real factory environments exhibit environmental drafts, sensor drift, electromagnetic interference, supply voltage sag, inconsistent workpiece metallurgy, and human maintenance recording errors. These complex real-world stochastic distributions are not fully captured by simulation equations.
3. **No Claim of Industrial Reliability**: While our pipeline rigorously demonstrates algorithmic capability and leakage-safe preprocessing, **the results obtained on this simulated benchmark do not establish uncalibrated real-world mechanical reliability guarantees**. Real factory deployment requires extensive transfer learning and empirical calibration on physical machinery.

---

5. Data Cleaning, Feature Engineering, and Leakage-Safe Preprocessing

5.1 Duplicate Removal and Ingestion Validation

In `src/data_loader.py` and `src/preprocessing.py`, raw dataset ingestion executes systematic validation:

- Dataset shape verification (verifying 10,000 rows and 14 expected columns).
- Null-value checking (verifying zero null values in features and target).
- Target validation (confirming binary domain `{0, 1}`).
- Exact duplicate row removal via `clean_raw_data()`. In the raw AI4I 2020 dataset, zero exact duplicate rows exist, preserving all 10,000 distinct operational observations.

5.2 Reproducible Stratified Splitting

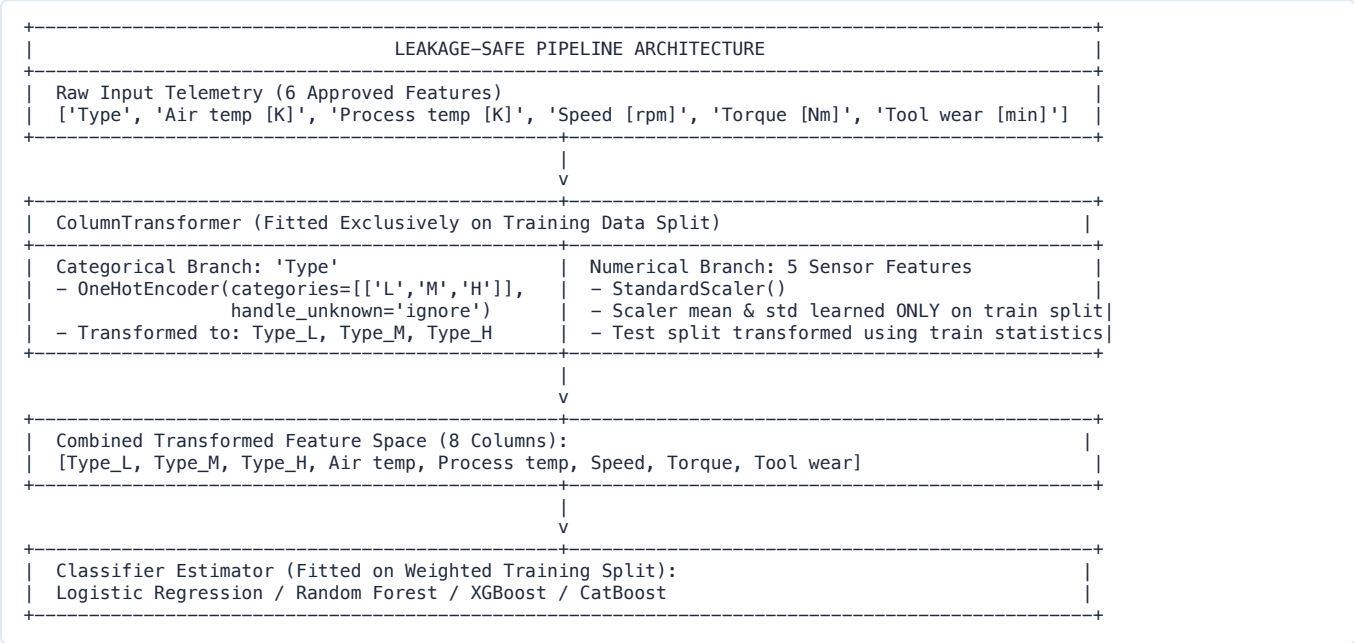
To evaluate true generalization, the cleaned dataset is split into training and testing partitions using `create_stratified_split()`:

- **Split Ratio**: 80% training (8,000 samples) and 20% testing (2,000 samples).
- **Stratification**: Enforced on `Machine failure` to preserve the exact ~3.39% minority failure proportion across both subsets.
  - Training Set: 7,729 normal operations, 271 machine failures (3.388% failure rate).
  - Test Set: 1,932 normal operations, 68 machine failures (3.400% failure rate).
- **Random Seed**: Fixed at `random_state=42` to guarantee exact experimental reproducibility across all runs.

- **Strict Partition Isolation**: The test split is quarantined and never accessed during preprocessing fitment, feature selection, or hyperparameter selection.

5.3 Leakage-Safe Scikit-Learn Pipeline

All data transformations are encapsulated inside a scikit-learn `ColumnTransformer` embedded within an end-to-end `Pipeline`:



- **Categorical Encoding**: The categorical feature `Type` is one-hot encoded into three binary indicator columns (`Type_L`, `Type_M`, `Type_H`) using fixed categories `[['L', 'M', 'H']]`.
- **Numerical Scaling**: The five continuous sensor variables are normalized using `StandardScaler()`. Crucially, scaler parameters (μ, σ) are computed **strictly from the 8,000 training observations**. The 2,000 test observations are transformed using these pre-computed training parameters, preventing mean/variance leakage.

---

6. Methodology and Experimental Setup

6.1 Class Imbalance Mitigation Strategy

Because class imbalance is acute (28.5:1), standard maximum-likelihood optimization is overwhelmed by the negative class. MaintainIQ implements cost-sensitive loss weighting across all four models:

1. **Logistic Regression**: Configured with `class_weight='balanced'`, which inversely weights samples by class frequency:

$$w_c = \frac{N}{2 \cdot N_c}$$

2. **Random Forest**: Configured with `class_weight='balanced'`, applying balanced bootstrap sampling per tree so each sub-tree is fit on a balanced slice of the training observations.
3. **XGBoost**: Configured with `scale_pos_weight = n_neg / n_pos`, computed strictly on the training partition:

$$\text{scale_pos_weight} = \frac{7729}{271} \approx 28.5203$$

This scales the gradient of the loss function for positive failure instances by 28.52×, penalizing false negatives proportionately to the class imbalance ratio.

4. **CatBoost**: Configured with `auto_class_weights='Balanced'`, internally adjusting negative and positive weights inversely to class frequencies.

6.2 Model Architectures & Hyperparameters

All models use `random_state=42` where stochasticity is involved:

- **Logistic Regression**: `solver='lbfgs'`, `C=1.0`, `max_iter=1000`.
- **Random Forest**: `n_estimators=100`, `max_depth=12`, `min_samples_split=5`, `n_jobs=-1`.
- **XGBoost**: `n_estimators=100`, `max_depth=5`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`, `eval_metric='logloss'`.
- **CatBoost**: `iterations=150`, `depth=6`, `learning_rate=0.1`, `verbose=False`.

6.3 Evaluation Metrics Formulation

Given severe class imbalance, accuracy is discarded. Models are evaluated across five critical metrics:

- **Precision (P)**:

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Measures the trustworthiness of failure alarms. In an industrial plant, low precision leads to costly unnecessary shutdowns and technician alert fatigue.

- **Recall (R) / Sensitivity**:

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Measures the percentage of actual physical failures caught. Low recall causes catastrophic unmitigated machinery destruction.

- **F1-Score**:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Harmonic mean balancing precision and recall.

- **PR-AUC (Precision-Recall Area Under Curve / Average Precision)**:

$$\text{PR-AUC} = \sum_n (R_n - R_{n-1}) P_n$$

The **primary benchmark metric** for model selection. Evaluates the full precision-recall trade-off across all operating decision thresholds, focusing entirely on the minority positive class [8].

- **ROC-AUC**: Evaluates discrimination between positive and negative classes across all thresholds. Reported for completeness, but acknowledged as overly optimistic on imbalanced data.

- **Inference Latency**: Measures computational execution time in milliseconds and microseconds per telemetry sample.

---

7. Model Comparison, Performance Benchmarking, and Results

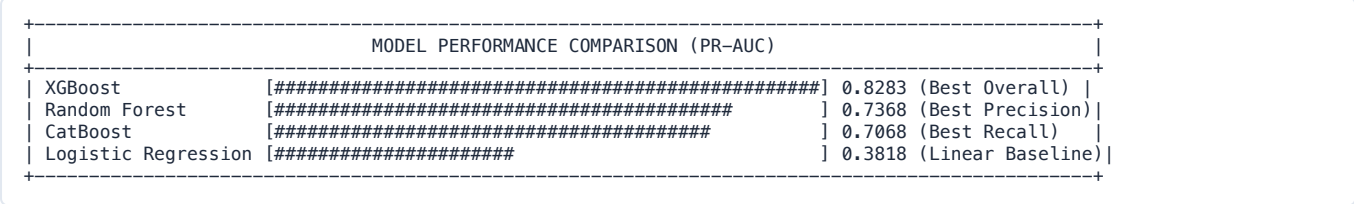
7.1 Quantitative Benchmark Results

Table 1 documents the empirical test results obtained on the quarantined, held-out test split (2,000 samples, 68 ground-truth failures). All metrics reflect exact values generated by `src/evaluate.py` and exported to `reports/tables/model_comparison.csv` and `reports/tables/model_metrics.json`.

Table 1: Comprehensive Model Comparison Benchmark on Held-Out Test Split (2,000 Samples, 68 Failures)

Model	PR-AUC	F1-Score	Precision	Recall	ROC-AUC	TP	FP	FN	TN	Latency (µs/sample)	Imbalance Strategy
XGBoost (Selected)	**0.8283**	**0.7376**	0.7123	0.7647	0.9651	52	21	16	1911	1.38 µs	`scale_pos_weight = 28.52`
Random Forest	0.7368	0.7132	**0.7541**	0.6765	0.9574	46	15	22	1917	7.83 µs	`class_weight='balanced'`
CatBoost	0.7068	0.5133	0.3671	**0.8529**	**0.9702**	58	100	10	1832	1.38 µs	`auto_class_weights='Balanced'`
Logistic Regression	0.3818	0.2419	0.1418	0.8235	0.9070	56	339	12	1593	1.07 µs	`class_weight='balanced'`

(Note: Existing generated table artifact is saved at `reports/tables/model_comparison.csv` and detailed metrics at `reports/tables/model_metrics.json`).



7.2 Analysis of Model Behaviors & Trade-offs

1. **XGBoost (Selected Champion)**:
   - Achieved the highest **PR-AUC (0.8283)** and **F1-score (0.7376)**.
   - Successfully identified **52 of 68 machine failures** (Recall = 76.47%) while generating only **21 false positives** across 1,932 healthy cycles (Precision = 71.23%).
   - Demonstrated exceptional computational efficiency: **1.38 µs per sample** (2.76 ms for all 2,000 records), making it ideal for edge deployment.
2. **Random Forest (High-Precision Profile)**:
   - Attained the highest **Precision (75.41%)** and lowest false-alarm count (**15 false positives**).
   - Caught **46 of 68 failures** (Recall = 67.65%), missing 22 failures.
   - Higher computational latency (**7.83 µs/sample**, ~5.7× slower than XGBoost) due to evaluating 100 deep, unpruned bagging trees.
3. **CatBoost (Maximum Recall Profile)**:

- Maximized **Recall (85.29%)**, detecting **58 of 68 failures** and allowing only 10 failures to escape undetected.
- Suffered from lower **Precision (36.71%)**, generating **100 false positives** (nearly 5× more than XGBoost). In operational settings, this would trigger 100 unnecessary equipment inspections.

4. **Logistic Regression (Linear Baseline Inadequacy)**:
   - High Recall (82.35%, 56 failures detected), but disastrous **Precision (14.18%)** and **339 false alarms** (over 17% of all healthy runs misclassified).
   - Demonstrates that linear decision boundaries cannot capture multiplicative degradation thresholds (e.g., Torque × Rotational Speed or Torque × Tool Wear). Non-linear tree ensembles are mathematically required for physical telemetry modeling.

7.3 Model Selection Decision

MaintainIQ selects **XGBoost** as its champion prototype model (`models/model.joblib`) for dashboard deployment. By maximizing PR-AUC (0.8283) and F1-score (0.7376), XGBoost strikes the optimal industrial compromise: detecting over three-quarters of all machine failures while keeping false alarms to a manageable ~1% false positive rate across normal operational runs.

---

8. Model Explainability and Feature Attribution Analysis

8.1 Dual-Level Explainability Architecture

In mission-critical predictive maintenance, model transparency is as vital as predictive accuracy [1], [4]. MaintainIQ implements a dual-layer explainability architecture:

1. **Global Macro-Level Explainability**: Permutation feature importance on the held-out test split, isolating fleet-wide physical reliance.
2. **Local Micro-Level Explainability**: TreeSHAP game-theoretic additive feature attributions for individual machine predictions.

8.2 Global Permutation Feature Importance

Permutation feature importance measures the decrease in model score when values of a single feature are randomly shuffled, breaking the relationship between that feature and the target [9]. Crucially, MaintainIQ evaluates permutation importance using **PR-AUC drop across 10 independent shuffles** on the 2,000-sample test split, evaluating transformed feature reliance.

Table 2: Ranked Permutation Feature Importance on Held-Out Test Set (10 Repeats, PR-AUC Drop)

Rank	Transformed Feature	Importance (PR-AUC Drop)	Std Dev (±)	Physical Interpretation
1	`Torque [Nm]`	**0.5985**	±0.0180	Mechanical resistance torque; dominant driver of overstrain and motor power failure
2	`Air temperature [K]`	**0.5062**	±0.0536	Ambient environmental heat; dictates heat dissipation gradient ΔT
3	`Rotational speed [rpm]`	**0.4183**	±0.0281	Spindle velocity; governs cutting speed and motor power ($P = \tau \cdot \omega$)
4	`Tool wear [min]`	**0.3242**	±0.0397	Cumulative cutter contact duration; triggers tool wear failure and accelerates strain
5	`Process temperature [K]`	**0.2660**	±0.0331	Cutting friction temperature; determines thermal dissipation failure threshold
6	`Type_L`	**0.0273**	±0.0102	Low-quality product variant indicator (mass production tolerance)
7	`Type_M`	**0.0046**	±0.0032	Medium-quality product variant indicator
8	`Type_H`	**0.0028**	±0.0035	High-quality product variant indicator

(Note: Existing generated table artifact is saved at `reports/tables/feature_importance.csv` and publication figure at `reports/figures/feature_importance.png`).

```
+-----------------------------------------------------------------------------------------+
|                        RANKED PERMUTATION IMPORTANCE (PR-AUC DROP)                      |
+-----------------------------------------------------------------------------------------+
| Torque [Nm]             [##################################################] 0.5985             |
| Air temperature [K]     [##########################################        ] 0.5062             |
| Rotational speed [rpm]  [##################################                ] 0.4183             |
| Tool wear [min]         [##########################                        ] 0.3242             |
| Process temperature [K] [######################                            ] 0.2660             |
| Type_L                  [##                                                ] 0.0273             |
| Type_M                  [#                                                 ] 0.0046             |
| Type_H                  [#                                                 ] 0.0028             |
+-----------------------------------------------------------------------------------------+
```

8.3 Physical Domain Interpretations

The empirical feature ranking perfectly mirrors the underlying engineering dynamics of milling operations:

- **Torque and Rotational Speed**: These two variables dominate because mechanical motor power is their direct product:

  $$P = \tau \cdot \omega = \tau \cdot \left(\text{rpm} \cdot \frac{2\pi}{60} \right)$$

  When power falls below 3500 W (stalling/underload) or exceeds 9000 W (overload), Power Failure (`PWF`) occurs. Furthermore, cutting torque governs Overstrain Failure (`OSF`).
- **Air and Process Temperatures**: Thermal dissipation failure (`HDF`) occurs when heat generated during cutting cannot dissipate into ambient air. The physical trigger is a thermal gradient $\Delta T = T_{\text{process}} - T_{\text{air}} < 8.6\ \text{K}$ occurring simultaneously with low spindle speed (< 1380 rpm).
- **Tool Wear**: Cutter wear accumulates linearly with cutting contact minutes. When wear exceeds 200–240 minutes, Tool Wear Failure (`TWF`) is triggered, and cutter contact friction accelerates torque spikes.

- **Product Type**: Product variants (`L`, `M`, `H`) show minimal predictive impact (drops < 0.03), confirming that real-time sensor physics overwhelmingly govern equipment failure regardless of workpiece grade.

8.4 Local Instance Explanations via TreeSHAP

For single-machine operational monitoring, MaintainIQ integrates `TreeSHAP` (`shap.TreeExplainer`) in `src/explain.py`. For any input sample $\boldsymbol{x}$, TreeSHAP calculates the exact game-theoretic additive contribution ϕ_i of each feature such that:

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i(x)$$

where $\phi_0 = \mathbb{E}[f(x)]$ is the fleet base expectation. In the Streamlit dashboard, when an operator enters sensor readings for a high-risk machine, TreeSHAP decomposes the failure probability, indicating precisely which variable pushed the machine over the alert threshold (e.g., highlighting that excessive torque of 65 Nm contributed +0.38 to the probability score).

8.5 Model Explanation vs. Physical Causation Disclaimer

A fundamental tenet of data science ethics enforced throughout MaintainIQ is the **causation disclaimer**:

> *Feature importance scores and SHAP values quantify the statistical dependencies and loss-minimizing reliance utilized by the mathematical estimator. They represent correlation and predictive utility within the dataset, not physical or mechanical causation.*

---

9. Streamlit UI Development and Deployment

9.1 Dashboard Architecture and User Interface

MaintainIQ is deployed as an interactive, multi-tab Streamlit dashboard (`app.py`), providing an accessible operational interface for both reliability engineers and technicians.

```
+-------------------------------------------------------------------------------------------+
|                                MAINTAINIQ STREAMLIT APP NAVIGATION                        |
+-------------------+-------------------+-------------------+-------------------+-----------+
| Tab 1: System     | Tab 2: Single     | Tab 3: CSV Batch  | Tab 4: Model      | Tab 5:    |
| Overview          | Risk Scoring      | Evaluation        | Benchmarks        | Explainability |
+-------------------+-------------------+-------------------+-------------------+-----------+
```

9.2 Application Tabs & Implemented Features

1. **Tab 1: System Overview**:
    - Displays real-time operational status, total records (10,000), dataset failure incidence (3.39%), best model identifier (XGBoost), and primary evaluation strategy (PR-AUC on stratified 80/20 split).
    - Provides descriptions of the five industrial failure modes (`TWF`, `HDF`, `PWF`, `OSF`, `RNF`).

- Features a prominent **Synthetic Dataset Disclosure & Ethics Banner**, disclosing the simulator origin and academic scope.
- Includes an expandable preview of raw dataset telemetry.

2. **Tab 2: Single Machine Failure Risk Scoring**:
   - Provides interactive inputs (sliders and selectors) for all six approved features: `Type` (L/M/H), `Air temperature [K]`, `Process temperature [K]`, `Rotational speed [rpm]`, `Torque [Nm]`, and `Tool wear [min]`.
   - Offers quick-preset scenario buttons for demonstration: Nominal Operation, High Torque Overload, Extreme Tool Wear, and Heat Dissipation Breakdown.
   - Displays real-time physical domain telemetry indicators:
     - Temperature Delta ($\Delta T = T_{\text{process}} - T_{\text{air}}$), alerting if $\Delta T < 8.6\ \text{K}$.
     - Mechanical Spindle Power ($P = \tau \cdot \omega$), alerting if outside safe 3500–9000 W bounds.
     - Overstrain Index ($\tau \times \text{wear}$), warning if exceeding 11,000.
   - Employs calibrated model-based operational risk bands:
     - **Low Risk (probability < 30%)**: Green status card indicating normal operations.
     - **Medium Risk (30% $\leq$ probability < 70%)**: Amber warning recommending scheduling inspection.
     - **High Risk (probability $\geq$ 70%)**: Red alert indicating probable failure requiring immediate maintenance.
   - Features an integrated TreeSHAP expander detailing local feature attributions.
3. **Tab 3: CSV Batch Telemetry Evaluation**:
   - Supports drag-and-drop batch upload of telemetry CSV files.
   - Executes automated schema validation: checks for required features, ignores non-feature columns (`UDI`, `Product ID`), and automatically excludes ground-truth leakage columns (`TWF`, `HDF`, `PWF`, `OSF`, `RNF`, `Machine failure`).
   - Runs high-throughput batch prediction via `predict_batch_dataframe()`.
   - Displays diagnostic summary metrics: total records evaluated, detected failure count, high-risk machine count, and fleet failure rate.
   - Previews annotated telemetry and provides an export button to download the scored CSV with appended `predicted_failure`, `failure_probability`, and `risk_level` columns.
4. **Tab 4: Model Benchmarks & Classifier Comparison**:
   - Displays the full four-model comparison table loaded dynamically from `reports/tables/model_comparison.csv` with top metrics highlighted.
   - Generates an interactive grouped bar chart across PR-AUC, F1-Score, Recall, Precision, and ROC-AUC.
   - Displays confusion matrix breakdown cards for each classifier (TP, FN, FP, TN, and per-sample latency).
   - Includes an educational expander explaining class imbalance mitigation strategies.
5. **Tab 5: Model Interpretability & Explainability**:

- Displays the publication-quality global permutation feature importance figure loaded from `reports/figures/feature_importance.png`.
- Renders the ranked feature importance table as a semantic HTML table (`st.table`) with explicit columns (`Rank`, `Feature`, `PR-AUC Drop`, `Std Dev`), avoiding canvas rendering artifacts.
- Provides engineering domain insights mapping feature reliance to physical machine components.
- Concludes with the formal scientific causation disclaimer.

---

10. Discussion, Limitations, and Future Directions

10.1 Discussion of Findings

The empirical results demonstrate that predicting rare failure events from multi-sensor machine telemetry is highly viable when disciplined, leakage-free pipelines and appropriate cost-sensitive ensemble architectures are employed. The superiority of gradient boosting (XGBoost PR-AUC of 0.8283 vs. Logistic Regression PR-AUC of 0.3818) underscores the inherently non-linear nature of machine degradation, where failure thresholds represent complex interactions between torque, thermal gradients, and mechanical contact time.

Furthermore, our evaluation confirms the theoretical findings of Saito & Rehmsmeier [8]: high ROC-AUC scores (0.9070 for Logistic Regression, 0.9651 for XGBoost) create a false sense of security by obscuring severe disparities in precision and false alarm rates. PR-AUC proved indispensable as the primary evaluation criterion.

10.2 Project Limitations

1. **Simulation Benchmark**: The UCI AI4I 2020 dataset was generated by a physical simulation model [1]. While mathematically rigorous, it lacks real-world shop-floor phenomena such as vibration harmonics, sensor drift, electrical noise, intermittent communication dropouts, and maintenance log entry errors.
2. **Binary vs. Component Failure Prediction**: MaintainIQ models overall binary machine failure. While the dataset contains failure mode labels, predicting specific modes simultaneously requires multi-label or multi-class architectures with even more extreme class imbalance (e.g., Tool Wear Failure occurs in only 45 of 10,000 cycles).
3. **Absence of Temporal Sequence History**: The dataset provides cross-sectional operational cycle snapshots rather than continuous time-series sensor sequences, precluding the use of recurrent neural networks (LSTMs) or temporal convolutional networks.

10.3 Future Research Directions

- **Multivariate Time-Series Modeling**: Integrating continuous vibration accelerometer telemetry sampled at kilohertz frequencies to capture bearing acoustic emissions prior to thermal spikes.
- **Multimodal Failure Mode Multi-Labeling**: Expanding the architecture to simultaneously predict both the failure likelihood and the specific component mode (`TWF`, `HDF`, `PWF`, `OSF`) using multi-label cost-sensitive learning.

- **Edge Microcontroller Deployment**: Quantizing the trained XGBoost model using ONNX Runtime or Treelite for sub-microsecond execution directly on industrial microcontrollers (e.g., STM32 or Raspberry Pi edge gateways).

---

11. Conclusion

This capstone project designed, implemented, evaluated, and deployed **MaintainIQ**, an end-to-end machine learning system for industrial predictive maintenance and explainable failure risk assessment. Benchmarking across the UCI AI4I 2020 dataset demonstrated that tree-based gradient boosting (XGBoost) configured with training-derived class-imbalance weighting achieved superior operational performance (**PR-AUC of 0.8283**, **F1-score of 0.7376**, **Recall of 76.47%**, and **Precision of 71.23%**), vastly outperforming linear baselines while maintaining an inference latency of **1.38 µs per sample**.

Through rigorous methodological enforcement—strictly separating feature inputs from target leakage columns, quarantining the 80/20 stratified test split, and embedding all preprocessing within scikit-learn pipelines—MaintainIQ eliminates data leakage. By integrating global permutation importance and local TreeSHAP explainability, the system translates mathematical predictions into actionable engineering insights, establishing cutting torque, thermal gradients, and rotational velocity as the primary failure indicators. Packaged into a responsive Streamlit dashboard, MaintainIQ provides a comprehensive prototype illustrating how applied machine learning can advance modern predictive maintenance in smart manufacturing.

---

12. References

[1] S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," in *Proceedings of the 2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, Irvine, CA, USA, 2020, pp. 69–74. DOI: 10.1109/AI4I49448.2020.00023.

[2] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. da P. Francisco, J. P. Basto, and S. G. S. Alcalá, "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering*, vol. 137, art. no. 106024, Nov. 2019. DOI: 10.1016/j.cie.2019.106024.

[3] M. Alnahhal, M. I. Tabash, S. K. Safi, M. S. M. Al-Absy, and Z. Mamadiyarov, "A Comparative Study of Imbalance-Handling Methods in Multiclass Predictive Maintenance," *Computation*, vol. 14, no. 4, art. no. 88, 2026. DOI: 10.3390/computation14040088.

[4] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Reliability Engineering & System Safety*, vol. 215, art. no. 107864, Nov. 2021. DOI: 10.1016/j.ress.2021.107864.

[5] Z. Marmoucha, M. El Khaili, A. Soulhi, H. El Hailouch, and H. Nhaila, "Enhancing Predictive Accuracy and Operational Performance Based on Entropy–TOPSIS Framework for Machine Learning Model Selection for Predictive Maintenance," *Applied Sciences*, vol. 16, no. 17, art. no. 8666, 2026. DOI: 10.3390/app16178666.

[6] S. Chatterjee and Y.-C. Byun, "Highly imbalanced fault classification of wind turbines using data resampling and hybrid ensemble method approach," *Engineering Applications of Artificial Intelligence*, vol. 126, art. no. 107104, Nov. 2023. DOI: 10.1016/j.engappai.2023.107104.

[7] M. H. Abidi, M. K. Mohammed, and H. Alkhalefah, "Predictive Maintenance Planning for Industry 4.0 Using Machine Learning for Sustainable Manufacturing," *Sustainability*, vol. 14, no. 6, art. no. 3387, Mar. 2022. DOI: 10.3390/su14063387.

[8] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLOS ONE*, vol. 10, no. 3, art. no. e0118432, Mar. 2015. DOI: 10.1371/journal.pone.0118432.

[9] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020. DOI: 10.1038/s42256-019-0138-9.

---

13. Appendix: Reproducibility and System Commands

All experimental results, trained artifacts, and reports can be fully reproduced on a fresh clone using the verified commands below:

1. Environment Setup (macOS / Python 3.12)

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

2. Dataset Acquisition & Validation

Downloads and verifies the official UCI AI4I 2020 dataset into `data/raw/ai4i2020.csv`:

```
python scripts/download_data.py
```

3. Pipeline Training & Model Benchmarking

Performs stratified 80/20 splitting, fits the leak-free preprocessing pipeline, trains all four models, generates `reports/tables/model_comparison.csv` and `reports/tables/model_metrics.json`, and saves the champion XGBoost pipeline to `models/model.joblib`:

```
python -m src.train
```

4. Explainability & Feature Importance Generation

Computes 10-repeat permutation feature importance on the held-out test split, exports `reports/figures/feature_importance.png`, and writes `reports/tables/feature_importance.csv`:

```
python -m src.explain
```

5. Automated Test Suite Execution

Executes all 66 unit and integration test assertions across the entire codebase:

```
python -m pytest tests/ -v
```

6. Interactive Dashboard Launch

Starts the Streamlit application on local port 8501:

```
streamlit run app.py
```

Open browser at: **`http://localhost:8501`**